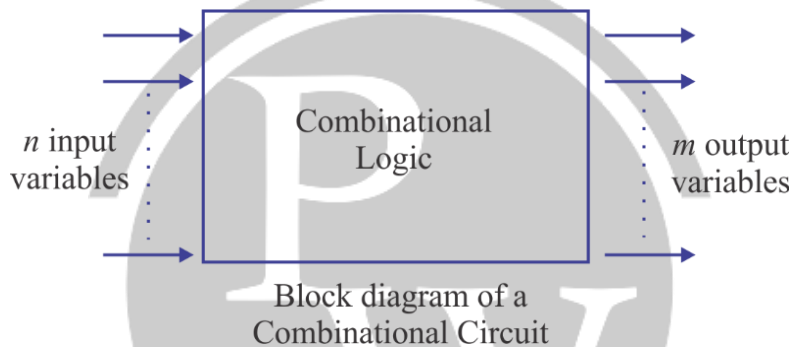


# 3

## COMBINATIONAL CIRCUITS

### 3.1. Combinational Circuits

The combinational circuit has ' $n$ ' input variables and ' $m$ ' output variables. Since, the number of input variables is  $n$ , there are  $2^n$  possible combinations of bits at the input. Each output can be expressed in terms of input variables by a Boolean expression.



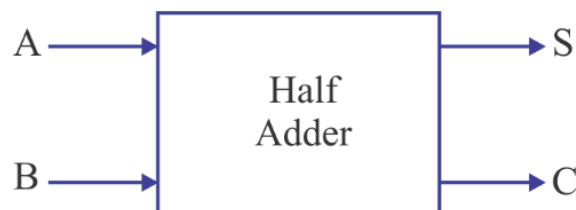
**Fig. 3.1. Block diagram of a Combinational Circuit**

### 3.2. Adders

The most basic arithmetic operation is the addition of two binary digits. A combinational circuit that performs the addition of two 1-bit numbers is called as half adder, and the logic circuit that adds three 1-bit numbers is called as full adder.

#### 3.2.1. Half Adder

The logic circuit that performs the addition of two 1-bit numbers is called as half adder. It is the basic building block for addition of two single bit numbers. This circuit has two outputs namely carry (C) and sum (S).



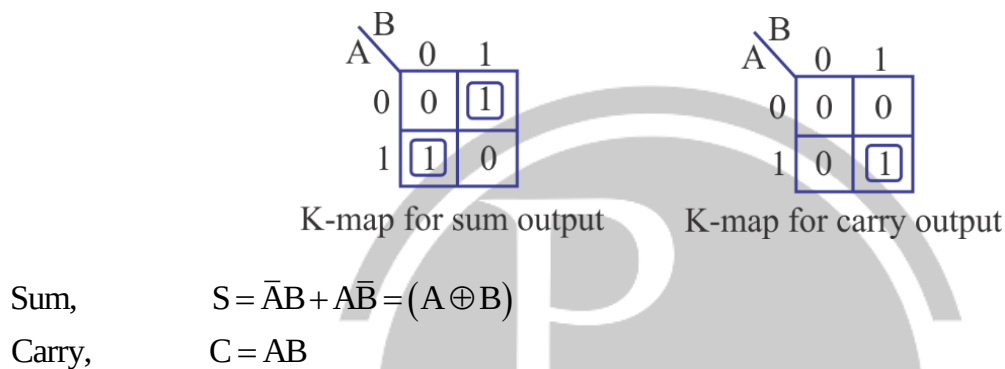
**Fig. 3.2. Block Diagram of a 2-bit Half Adder**

**The truth table of half adder:** where A and B are the inputs and sum and carry

**Table 1: Truth Table of Half Adder**

| Inputs |   | Outputs |           |
|--------|---|---------|-----------|
| A      | B | Sum (S) | Carry (C) |
| 0      | 0 | 0       | 0         |
| 0      | 1 | 1       | 0         |
| 1      | 0 | 1       | 0         |
| 1      | 1 | 0       | 1         |

**K-map simplification for Carry and Sum:** Boolean expressions for the sum (S) and carry (C) outputs from K – maps:



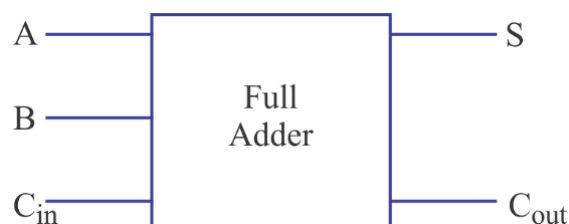
**Logic Diagram:**



**Fig. 3.3. Logic Diagram of Half Adder**

### 3.2.2. Full Adder

A full adder circuit is an arithmetic circuit block that can be used to add three bits to produce a sum and a carry output. Let us consider A and B as two 1-bit inputs &  $C_{in}$  is a carry generated from the previous order bit additions. Let S (sum) and  $C_{out}$  (carry) are the outputs of the full adder.



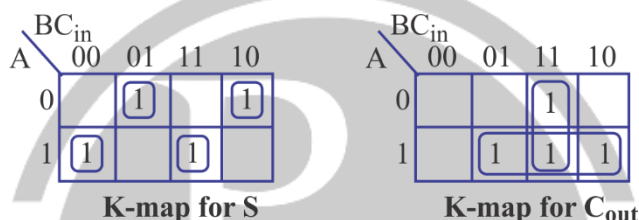
**Fig. 3.4. Block Diagram of a Full Adder**

The Truth Table for Full Adder is given as:

Table: Truth Table for Full Adder

| Inputs |   |                 | Outputs |                        |
|--------|---|-----------------|---------|------------------------|
| A      | B | C <sub>in</sub> | Sum (S) | Carry C <sub>out</sub> |
| 0      | 0 | 0               | 0       | 0                      |
| 0      | 0 | 1               | 1       | 0                      |
| 0      | 1 | 0               | 1       | 0                      |
| 0      | 1 | 1               | 0       | 1                      |
| 1      | 0 | 0               | 1       | 0                      |
| 1      | 0 | 1               | 0       | 1                      |
| 1      | 1 | 0               | 0       | 1                      |
| 1      | 1 | 1               | 1       | 1                      |

K – map Simplification for Carry and Sum:



Simplified Boolean expressions for the sum (S) and carry (C<sub>out</sub>) output from K-maps is

$$\begin{aligned}
 \text{Sum, } S &= \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + AB\bar{C}_{in} + A\bar{B}C_{in} = C_{in}(\bar{A}\bar{B} + AB) + \bar{C}_{in}(\bar{A}B + A\bar{B}) \\
 &= C_{in}(A \odot B) + C_{in}(A \oplus B) \\
 &= C_{in}(\overline{A \oplus B}) + C_{in}(A \oplus B)
 \end{aligned}$$

$$\text{Sum, } S = C_{in} \oplus A \oplus B$$

$$\text{Carry, } C_{out} = AB + AC_{in} + BC_{in}$$

**Logic Diagram:**

We can realize logic diagram of a full adder using gates as shown in below figure:

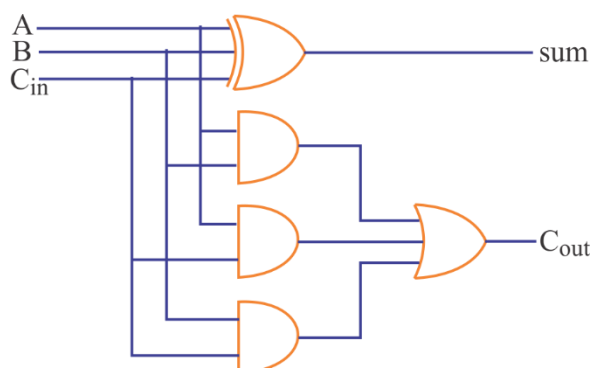


Fig. 3.6. Logic Diagram of Full Adder

**Example:** A full adder is implemented using two input OR gate and two half adders. Half adder is implemented using two input XOR and two input AND gate. The propagation delays of XOR gate, AND gate and OR gate respectively are 2ns, 1.5ns, and 1ns. The propagation delay of full adder is ..... ns.

**Solution:**

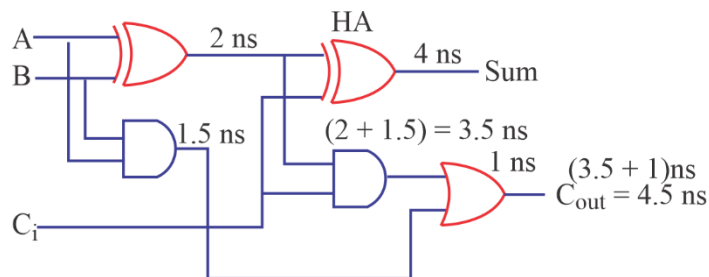


Fig. 3.7.

### 3.3. Subtractors

#### 3.3.1. Half subtractor

A half subtractor is a combinational logic circuit, which performs the subtraction of two 1-bit numbers. It subtracts one binary digit from another to produce a DIFFERENCE output and a BORROW output.

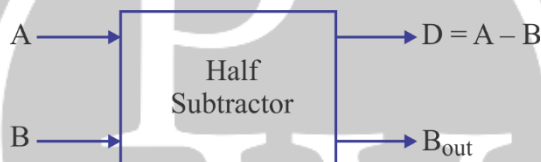


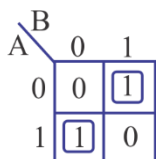
Fig. 3.8. Block diagram of a half subtractor

**The truth table of half – subtractor**, where A, B are the inputs, and difference (D) and borrow (B) are the outputs.

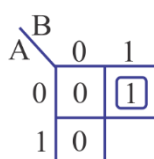
Table: Truth table of Half – Subtractor

| Inputs |   | Outputs |                  |
|--------|---|---------|------------------|
| A      | B | D       | B <sub>out</sub> |
| 0      | 0 | 0       | 0                |
| 0      | 1 | 1       | 1                |
| 1      | 0 | 1       | 0                |
| 1      | 1 | 0       | 0                |

**K – map Simplification for Difference and Borrow:**



K-map for difference output



K-map for borrow output

Difference,  $D = \bar{A}B + A\bar{B} = A \oplus B$

Borrow,  $B_{out} = \bar{A}B$

### Logic Diagram:

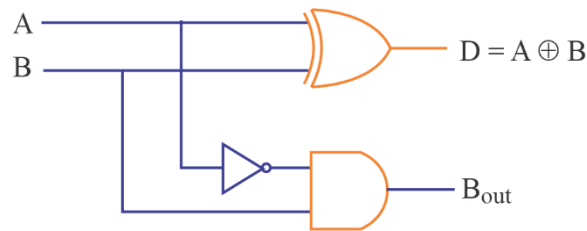


Fig. 3.9. Logic Diagram of a Half subtractor

### 3.3.2. Full Subtractor

A full subtractor performs subtraction operation on two bits, a minuend and a subtrahend.

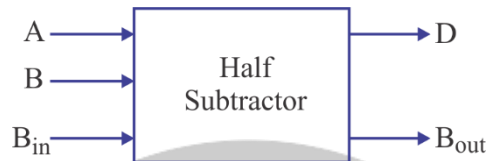


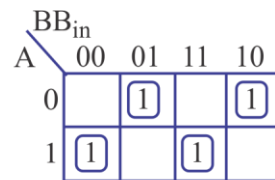
Fig. 3.10. Block Diagram of a Full subtractor

### Truth table of Full subtractor:

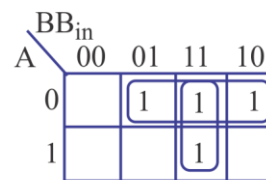
Table: Truth table of Full subtractor

| Inputs |   |                 | Outputs |                  |
|--------|---|-----------------|---------|------------------|
| A      | B | B <sub>in</sub> | D       | B <sub>out</sub> |
| 0      | 0 | 0               | 0       | 0                |
| 0      | 0 | 1               | 1       | 1                |
| 0      | 1 | 0               | 1       | 1                |
| 0      | 1 | 1               | 0       | 1                |
| 1      | 0 | 0               | 1       | 0                |
| 1      | 0 | 1               | 0       | 0                |
| 1      | 1 | 0               | 0       | 0                |
| 1      | 1 | 1               | 1       | 1                |

### K – map simplification for Difference and Borrow:



K-map for difference output



K-map for borrow output

Difference,

$$\begin{aligned}
 D &= \bar{A}\bar{B}B_{in} + \bar{A}B\bar{B}_{in} + A\bar{B}\bar{B}_{in} + AB B_{in} \\
 &= B_{in} (AB + \bar{A}\bar{B}) + \bar{B}_{in} (\bar{A}\bar{B} + AB) \\
 &= B_{in} (\overline{A\bar{A}}) + \bar{B}_{in} (A\bar{A}B)
 \end{aligned}$$

$$D = A \oplus B \oplus B_{in}$$

Borrow,

$$B_{in} = \bar{A}B + \bar{A}B_{in} + BB_{in}$$

## Logic Diagram

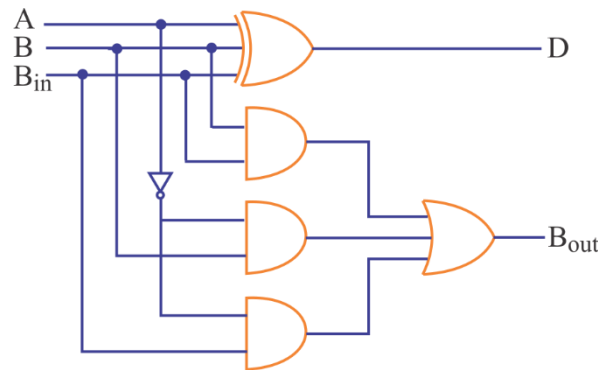


Fig. 3.11. Logic Diagram of a Full Subtractor

## 3.4. Binary Parallel Adder

An n-bit parallel adder can be constructed using n number of full adders are connected in parallel and hence; it is also known as parallel adder such that the previous carry or carry input for adder 0 is set to zero. The carry output of each adder is connected to the carry input of the next higher order adder. Hence, it is also known as carry propagate adder.

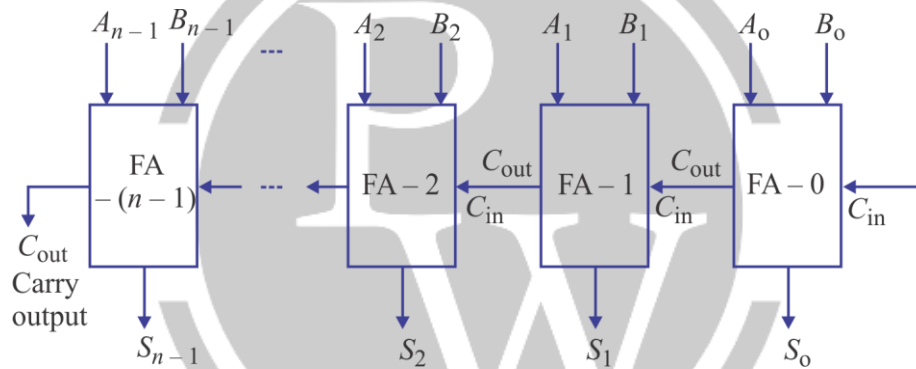


Fig. 3.12. n-bit Binary Adder

### 3.4.1. Propagation Delay in Parallel Adder:

Parallel adders suffer from propagation delay problem because higher bit additions depend on the carry generated from lower bit addition. In effect, carry bits must propagate or ripple through all stages before the most significant sum bit is valid. Thus, the total sum (the parallel output) is not valid until after the cumulative delay of all the adder.

## 3.5. Carry Look Ahead Adder

The look ahead carry adder speeds up the operation by eliminating this ripple carry delay. It examines all the input bits simultaneously and also generates the carry in bits for all the stages simultaneously. The method of speeding up the addition process is based on the two additional functions of the full adder called the carry generate and carry propagate functions.

### 3.5.1. Carry Generation

Carry is generated only if both the input bits are 1, that is, if both the bits A and B are 1's, a carry has to be generated in this stage regardless of whether the input carry  $C_{in}$  is a 0 or a 1. Let G as the carry generation function,

$$G = A \cdot B$$

Consider the present bit as the  $n^{th}$ , then

$$G_n = A_n \cdot B_n$$

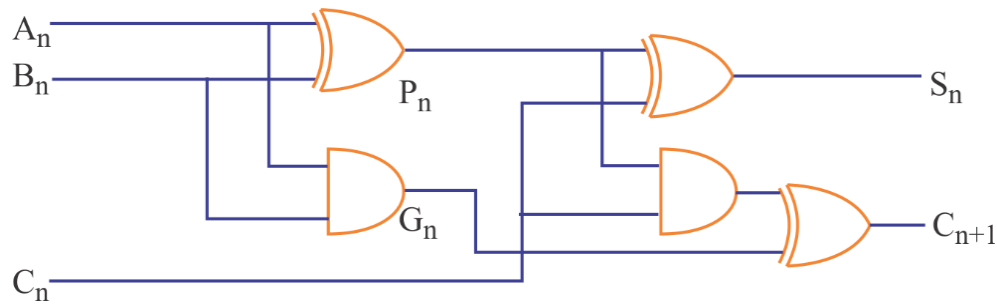


Fig. 3.13. Carry Look – ahead Generator Circuit

### 3.5.2. Carry Propagation

A carry is propagated if any one of the two input bits A or B is 1. If both A and B are 0, a carry will never be propagated. On the other hand, if both A and B are 1, then will not propagate the carry but will generate the carry. Let P as the carry – propagation function, then

$$P_n = A_n \oplus B_n$$

### 3.5.3. Look ahead Expressions

Let  $n^{\text{th}}$  bit adder, the sum (S) and the carry out (C) for the  $n^{\text{th}}$  bit may be expressed in terms of the carry generation function (G) and the carry propagation function (P) as

$$S_n = P_n \oplus C_n$$

$$C_{n+1} = G_n + P_n \cdot C_n$$

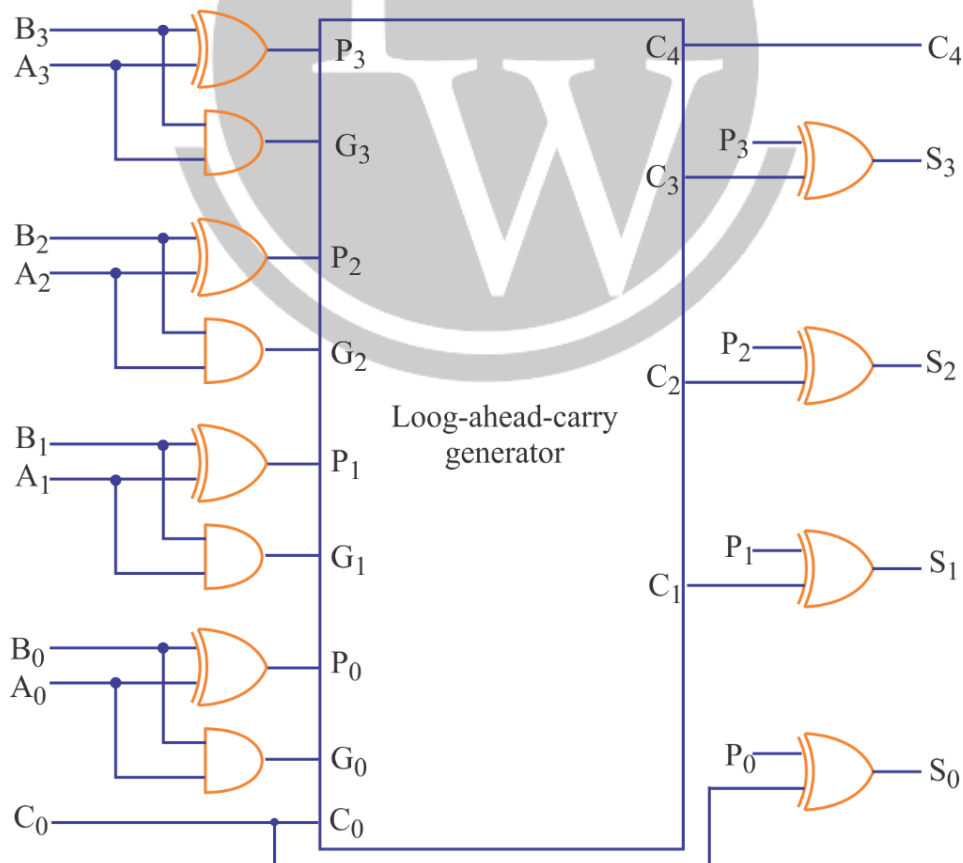


Fig. 3.14. 4-bit Full Adder with a look Ahead Carry Generator

**Example:** A full adder can be realized using half adder? Explain it in detail.

**Solution:** A full adder realization using half adder is given by:

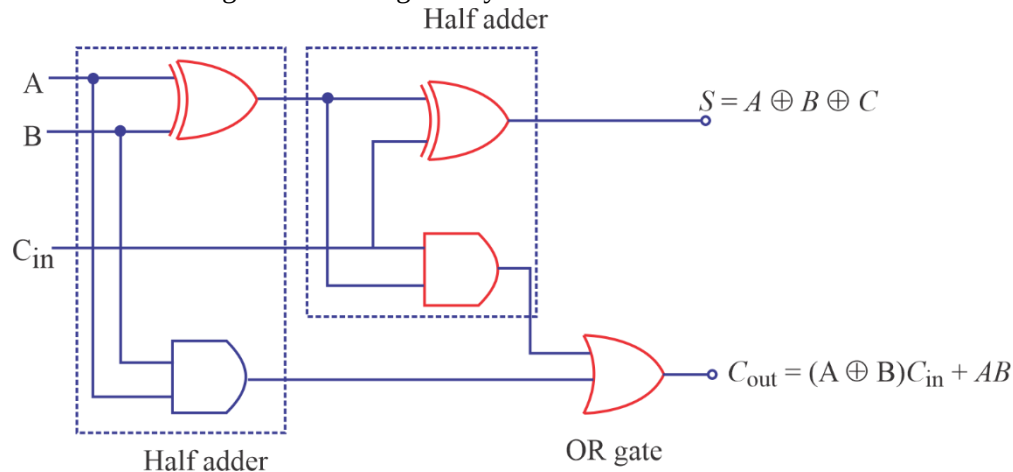


Fig. 3.15.

### 3.6. Comparator

The comparator is a combinational logic circuit. It compares the magnitude of two  $n$ -bit numbers and provides the relative result as the output. Let  $A$  and  $B$  are the two  $n$ -bit inputs. The comparator has three outputs namely  $A > B$ ,  $A = B$  and  $A < B$ . Depending upon the result of comparison, one of these outputs will go high.

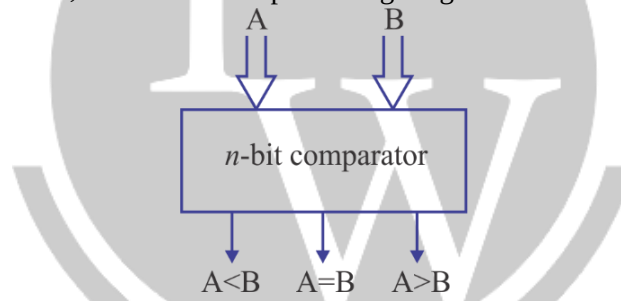


Fig. 3.16. Block diagram of digital comparator

#### 3.6.1. 1-bit Magnitude Comparator

The 1-bit comparator is a combinational logic circuit with two inputs  $A$  and  $B$  and three outputs namely  $A < B$ ,  $A = B$  and  $A > B$ .

Table : Truth Table of a 1-bit Comparator

| Inputs |   | Outputs       |               |               |
|--------|---|---------------|---------------|---------------|
| A      | B | X ( $A < B$ ) | Y ( $A = B$ ) | Z ( $A > B$ ) |
| 0      | 0 | 0             | 1             | 0             |
| 0      | 1 | 1             | 0             | 0             |
| 1      | 0 | 0             | 0             | 1             |
| 1      | 1 | 0             | 1             | 0             |

**Design of 1-bit Magnitude Comparator:** We can write the expressions for the three outputs as under: